



UNIVERSIDADE FEDERAL DE SANTA CATARINA
CENTRO TECNOLÓGICO CIENTÍFICO
DEPARTAMENTO DE ENGENHARIA ELÉTRICA
E ELETRÔNICA



CTC UFSC
EEL
Departamento de Engenharia
Elétrica e Eletrônica

Algoritmo de Reconhecimento de Resistores Baseado em Visão Computacional

Heron Vitor Macedo Gerati (22103494)
EEL7815 – Projeto Nível I em Controle e Processamento de Sinais I

Florianópolis, Julho de 2026.

Sumário

1	Introdução	2
2	Fundamentação Teórica	3
2.1	Transformada da Distância	3
2.2	Espaço de Cor HSV	3
3	Projeto	5
3.1	Limitações	5
3.2	Fluxo de projeto	5
3.2.1	Segmentação do corpo do resistor	6
3.2.2	Segmentação das faixas coloridas	9
3.2.3	Ordenação das faixas	10
3.2.4	Decisão das cores e determinação da resistência	12
3.2.5	Preparação da imagem de saída	12
4	Resultados	14
5	Conclusão	15

1 Introdução

Resistores são componentes eletrônicos muito pequenos utilizados em eletrônica analógica geral. Devido ao seu tamanho reduzido é de grande dificuldade a impressão do valor de sua resistência diretamente no corpo do resistor [1]. Tendo em vista esse fator, criou-se o código de cores de resistores. Esse código atribui um certo valor de resistência a faixas coloridas que são impressas no corpo de resistores. Para resistores de 4 faixas, segue a equação abaixo:

$$R = (C_1 \times 10 + C_2) \times 10^{C_3} \pm C_4\%, \quad (1)$$

na qual C_1 é o valor numérico da primeira faixa, C_2 da segunda faixa, C_3 da terceira faixa e C_4 o valor da tolerância referente a quarta faixa. A Figura 1 apresenta exemplos de resistores de 4 faixas, nesse caso, todas com a quarta faixa dourada, atribuindo uma tolerância de 5%.

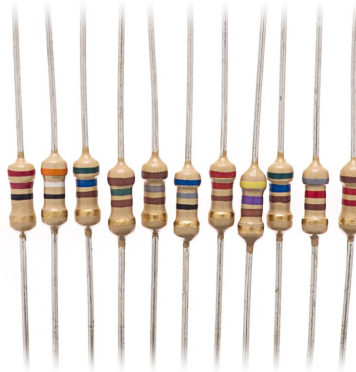


Figura 1: Exemplo de resistores de 4 faixas coloridas. Fonte: [wjcomponentes](#)

Resistores também são um dos principais componentes utilizados em laboratórios de eletrônica de universidades de todos o mundo, devido ao fato de ser um dos primeiros componentes e parâmetros (resistência) estudados, além de ser um componente fundamental no desenvolvimento de qualquer circuito. Um dos principais desafios enfrentados por esses laboratórios é justamente a organização desses componentes, afinal, muitas vezes aqueles que frequentam e utilizam desses espaços para seus estudos não se preocupam com o estado do local. Os problemas são diversos, variando desde componentes armazenados em locais incorretos até componentes danificados sendo confundidos com peças em perfeito funcionamento. Tendo em vista os problemas enfrentados, propõe-se o projeto desenvolvido para a disciplina: um algoritmo capaz de detectar a resistência de um resistor a partir de apenas uma foto sua. Para isso o algoritmo irá analisar a cor de cada faixa do resistor e sua posição no corpo e determinará o valor da resistência.

O projeto desenvolvido propõe o desenvolvimento de um algoritmo utilizando apenas técnicas clássicas de visão computacional, não utilizando nenhuma técnica de aprendizado de máquina como redes neurais, etc. A escolha dessa metodologia é dada com a justificativa de reforçar o aprendizado das técnicas clássicas por parte do autor, além de que, um algoritmo que não dependa de uma rede neural para seu funcionamento pode ser capaz de executar suas iterações mais rapidamente.

O algoritmo foi inteiramente desenvolvido na linguagem Python utilizando-se de diversas bibliotecas (como numpy, sys, etc) mas principalmente a biblioteca do OpenCV [2] que tem implementada todas as ferramentas

estudadas no decorrer da disciplina.

2 Fundamentação Teórica

Nesta seção serão comentados sobre alguns conceitos e técnicas aplicadas no trabalho que não foram comentadas em aula.

2.1 Transformada da Distância

A Transformada da Distância (*Distance Transform*) é uma técnica de processamento digital de imagens utilizada para calcular, em cada pixel de uma imagem binária, a distância até o pixel mais próximo pertencente a uma região de referência. Em uma imagem binária, os pixels são classificados como pertencentes ao objeto ou ao fundo, e a transformada gera uma nova imagem em que cada pixel armazena a menor distância até a região oposta [3].

Matematicamente, para um pixel p , a transformada é definida como

$$D(p) = \min_{q \in S} d(p, q), \quad (2)$$

onde S representa o conjunto de pixels de referência e $d(p, q)$ é uma métrica de distância. A métrica mais utilizada é a distância Euclidiana, definida por

$$d(p, q) = \sqrt{(p_x - q_x)^2 + (p_y - q_y)^2}. \quad (3)$$

Além da distância Euclidiana, podem ser utilizadas métricas alternativas, como a distância Manhattan (*city-block*), a distância *chessboard* e aproximações do tipo *chamfer*. A escolha da métrica influencia diretamente a representação das distâncias e a complexidade computacional do algoritmo empregado.

A Transformada da Distância possui diversas aplicações em visão computacional e processamento de imagens, incluindo segmentação de objetos, geração de diagramas de Voronoi, extração de esqueletos (*skeletonization*), determinação de centros geométricos e separação de objetos sobrepostos. Em muitos casos, os máximos locais da transformada correspondem aos pontos mais centrais das regiões analisadas, tornando-a uma ferramenta importante para análise de forma e reconhecimento de padrões [3].

2.2 Espaço de Cor HSV

O espaço de cor HSV (*Hue, Saturation, Value*) é um modelo de representação de cores amplamente utilizado em aplicações de processamento digital de imagens e visão computacional. Diferentemente do modelo RGB, no qual as cores são representadas diretamente pelas intensidades dos canais vermelho, verde e azul, o modelo HSV separa a informação cromática da informação de luminosidade, permitindo uma representação mais próxima da percepção humana das cores [2].

Nesse modelo, o componente *Hue* (H) representa a tonalidade da cor, sendo normalmente associado à posição angular de uma cor em um círculo cromático. O componente *Saturation* (S) descreve o grau de pureza ou intensidade da cor, enquanto o componente *Value* (V) representa o brilho da imagem. Essa decomposição

permite analisar características cromáticas independentemente das variações de iluminação presentes na cena. As Figuras 2 e 3 apresentam representações visuais dos espaços HSV.

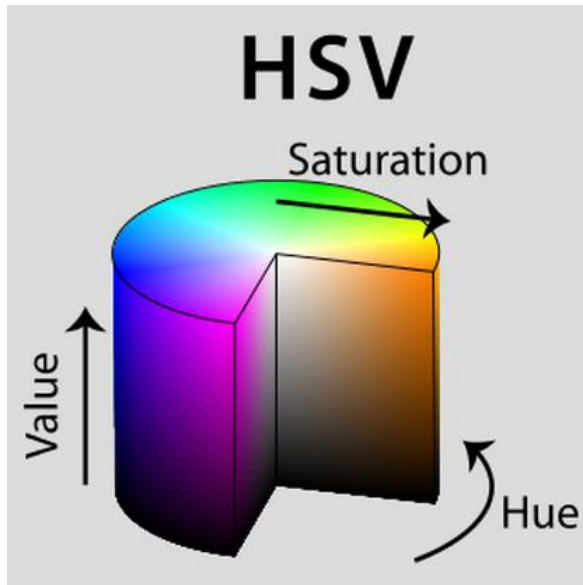


Figura 2: Representação visual 1 do espaço HSV. Fonte: [StackOverflow](#)

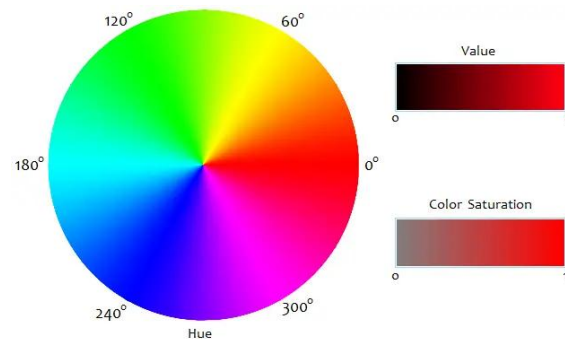


Figura 3: Representação visual 1 do espaço HSV. Fonte: [Had2Know](#)

A principal vantagem do espaço HSV em aplicações de visão computacional está na facilidade de segmentação de cores. Como a tonalidade é armazenada separadamente do brilho, é possível identificar objetos coloridos de forma mais robusta mesmo quando ocorrem alterações na iluminação do ambiente. Por esse motivo, o modelo HSV é frequentemente empregado em tarefas de detecção de objetos, rastreamento visual, classificação de imagens e extração de características cromáticas [2].

3 Projeto

Nesta seção serão comentados sobre as limitações do algoritmo, sobre o fluxo de projeto utilizado para desenvolver o algoritmo e sobre cada etapa e as estratégias utilizadas para resolver os problemas encontrados em cada uma.

3.1 Limitações

Como comentado, o algoritmo não faz uso de técnicas de aprendizado de máquina para determinar o valor da resistência do componente. Apesar disso acelerar a execução do algoritmo isso também diminui a robustez do programa. As estratégias utilizadas para segmentação do resistor, das faixas do código de cores e da determinação das cores dependem fortemente das condições na qual a foto foi tirada, ou seja, caso a foto não seja adequada, o algoritmo simplesmente não funciona. Das limitações impostas para o funcionamento do código, são elas:

- Fundo branco: a estratégia utilizada para separar o componente do fundo da foto (para que seja possível trabalhar apenas na região do corpo do resistor - a região ótima onde se encontram as faixas) depende que o fundo da imagem seja branco. Inicialmente, testaram-se técnicas de limiarização com fundos de outras cores, assim como com fundos texturizados. Devido as características do próprio resistor (cor, forma) a segmentação falhou e resolveu-se adotar o fundo branco como pré-requisito que pode ser facilmente alcançado colocando o resistor sob uma folha A4 antes de fotografá-lo.
- Iluminação: a determinação de certas cores do código do resistor (cinza, preto, etc) dependem da luminosidade da foto. Como o algoritmo foi desenvolvido para ser usado em ambientes como laboratórios, utilizou-se como referência fotos tiradas nesse tipo de ambiente para auxiliar nos ajustes do decisor de cores. Dessa forma, fotos tiradas com iluminação baixa, das quais não é possível diferenciar com clareza cores mais escuras podem não funcionar.
- Sombras: como comentado, o limiarizador parte do pressuposto de que o fundo da foto será branco. Portanto, caso haja alguma sombra em torno do corpo do resistor, escurecendo o fundo, é possível que essa região seja tratada como parte do corpo atrapalhando na segmentação das faixas. Entretanto, sombras na região das pernas do resistor não interferem no funcionamento, o mesmo procedimento que irá eliminar as pernas do resistor durante a análise é também capaz de eliminar a sombra das pernas.
- Faixa de tolerância: Como a cor da faixa de tolerância é muitas vezes semelhante a do corpo do resistor, ele falha em encontrar essa faixa. Portanto mesmo que o algoritmo encontre esse faixa ele irá ignorá-la para que o funcionamento do algoritmo não seja limitado aos casos em que a faixa não é encontrada. Entendeu-se que isso não seria um problema de grande importância já que, nos laboratórios comentados, os resistores muitas vezes são separados apenas pelo valor da resistência que pode ser calculado apenas com as 3 primeiras faixas.

3.2 Fluxo de projeto

O algoritmo funciona de forma simples para o usuário. Ele entre com uma imagem de um resistor e o algoritmo devolve uma imagem do resistor reorientado no sentido de leitura, com as faixas do código de cores

destacadas e com o valor da resistência escrito acima do resistor. Para alcançar o resultado desejado, seguiu-se um desenvolvimento sequencial do algoritmo. O fluxograma do projeto pode ser visto na Figura 4.

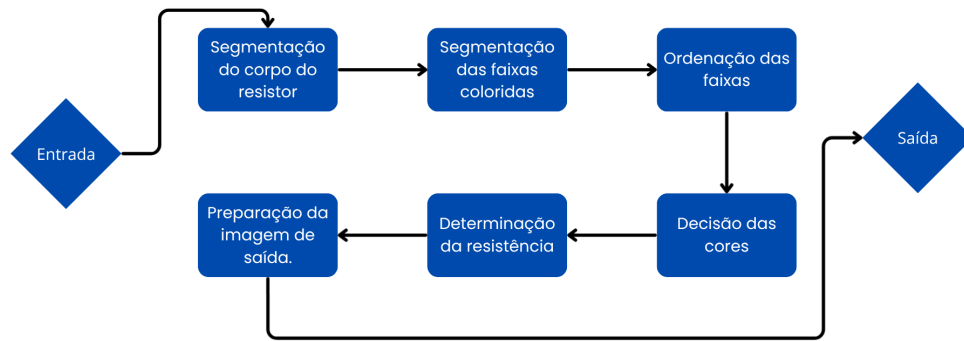


Figura 4: Fluxograma de desenvolvimento do algoritmo de determinação de resistência. Fonte: Autor.

Cada bloco do fluxograma será comentado a seguir.

3.2.1 Segmentação do corpo do resistor

O primeiro desafio do algoritmo é separar o resistor do fundo da imagem, assim como remover suas pernas. O objetivo é trabalhar apenas na região em que se encontram as faixas coloridas, ou seja, o corpo do resistor. Para alcançar tal objetivo, construiu-se uma máscara formada por pixels brancos em toda a região do corpo do resistor e por pixels pretos em toda região do fundo. Em um momento inicial, pensou-se na utilização de algum algoritmo de limiarização adaptativa. Esses algoritmos são capazes de separar um objeto do fundo utilizando como base seus pixels vizinhos. Havendo um sucesso na utilização dessa técnica o algoritmo se torna mais genérico, devido a própria natureza da limiarização adaptativa. As Figuras 5 e 6 apresentam resultados alcançados utilizando essa técnica.

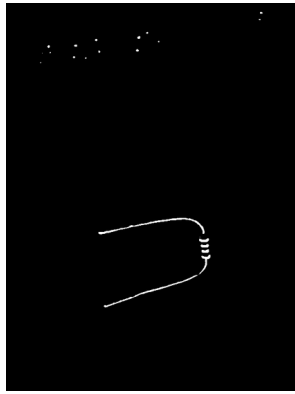


Figura 5: Resultado alcançado com limiarização adaptativa mais agressiva.

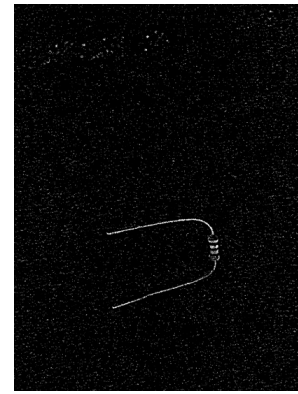


Figura 6: Resultado alcançado com limiarização adaptativa menos agressiva.

Observando as Figuras, percebe-se que nenhuma das duas teve sucesso em separar o resistor como um único objeto do fundo da fotografia. No caso da Figura 6 o resultado ainda apresentou uma grande quantidade de ruído. Buscou-se eliminar o ruído e juntar todo o corpo do resistor em um só objetivo aplicando operações morfológicas. As Figuras 7 e 8 apresentam o resultado após a aplicação de uma iteração de erosão e seis de dilatação.

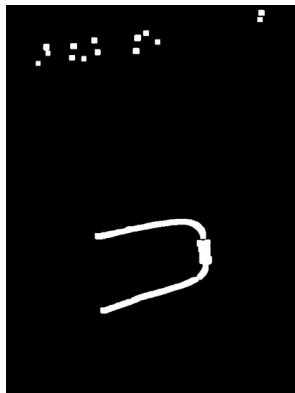


Figura 7: Resultado alcançado após operações morfológicas.



Figura 8: Resultado alcançado após operações morfológicas.

Analisando as Figuras, percebe-se que, mesmo após a aplicação das operações comentadas o resultado ainda não é ideal. No primeiro caso, obteve-se o resistor. Entretanto, as pernas ficaram muito grossas o que dificulta em sua remoção da máscara obtida. Na segunda o resistor foi perdido totalmente na aplicação da operação de erosão.

Para contornar os resultados obtidos, como o fundo e as condições das imagens já estavam estabelecidas, decidiu-se construir a máscara utilizando um limiarizador convencional. Utilizando imagens de referência, obteve-se os valor dos limiares e construiu-se uma nova máscara, que pode ser visualizada na Figura 9.

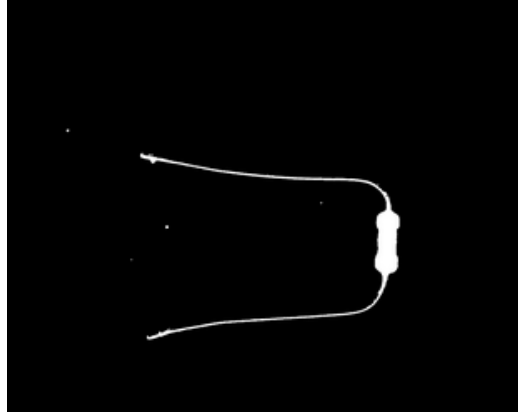


Figura 9: Máscara final obtida com limiarização convencional.

Percebe-se que o resultado alcançado é fortemente superior aos anteriores.

Na sequência, para remover as pernas da máscara, utilizou-se do algoritmo da transformada da distância, comentado anteriormente. O objetivo era obter um valor que fosse menor do que as dimensões do corpo do resistor mas maior do que as dimensões das pernas. Com esse valor obtido, constrói-se um kernel quadrado com lado igual a esse valor e aplica uma abertura morfológica sobre a imagem.

Aplicando a transformada na imagem ela irá retornar a distância de cada pixel branco até o pixel preto mais próximo. Extraíndo o máximo global para toda máscara, espera-se obter o valor da distância de algum ponto mais central do corpo do resistor até o fundo preto, esse tamanho é exatamente o valor que estava sendo procurado para a construção do kernel. O resultado após aplicação da abertura com o elemento obtido é apresentado na Figura 10



Figura 10: Máscara final com as pernas removidas.

Percebe-se que as pernas foram perfeitamente removidas, sobrando apenas a região desejada.

Com o corpo do resistor extraído, antes de seguir para etapa de segmentação das faixas do código de cores, o algoritmo irá rotacionar a imagem, de forma a deixar o resistor na horizontal. Todo o processamento após essa etapa será realizado com o resistor reorientado.

3.2.2 Segmentação das faixas coloridas

Para a extração das regiões das faixas coloridas também serão construídas máscaras. Entretanto, cada uma utilizará critérios diferentes em sua construção.

Inicialmente, obtém-se uma imagem somente com a região do corpo do resistor aplicando a máscara obtida na etapa anterior. Em sequência, converte-se o resultado para o espaço HSV, comentado anteriormente. Dos pixels do corpo do resistor, extrai-se os valores H e obtém-se o histograma desses valores. A Figura 11 apresenta um exemplo de histograma obtido.

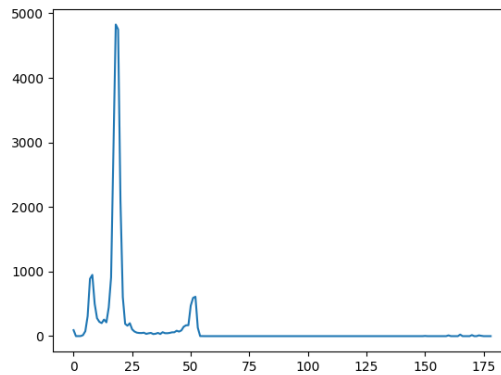


Figura 11: Exemplo de histograma obtido pelo algoritmo. Eixo X representa o valor de H e eixo Y representa o número de amostras.

Analisando o histograma, percebe-se que há um valor de H que se repete consideravelmente mais do que os outros. Esse valor representa a cor que mais se repete no corpo do resistor, ou seja, a cor do próprio corpo (por exemplo, a cor bege de alguns resistores). O algoritmo nunca plota esse histograma e o devolve ao usuário. Entretanto, ele identifica automaticamente o valor de H mais frequente e o utiliza para construir duas máscaras binárias.

A primeira máscara seleciona os pixels cujo valor de H se encontra dentro de uma faixa de tolerância em torno do valor predominante, correspondendo majoritariamente ao corpo do resistor. A segunda máscara é complementar à primeira, destacando os pixels cujos valores de H se encontram fora dessa faixa, o que tende a evidenciar as faixas coloridas presentes no componente.

A segunda máscara é a base para a identificação das faixas coloridas. Entretanto, como ela é construída apenas a partir do valor de H, pode acabar eliminando pixels de faixas cuja matiz é muito próxima à do corpo do resistor. Dessa forma, a primeira máscara, que indica a posição dos pixels filtrados, é utilizada para recuperar possíveis pixels de faixas removidos erroneamente.

Primeiramente, deseja-se recuperar os pixels de cores mais vibrantes. Escolheu-se seguir essa estratégia, pois, de forma geral, as cores das faixas dos resistores costumam ser mais vivas do que a cor do corpo, justamente para que possam ser facilmente distinguidas até mesmo pela visão humana. Cores desse tipo podem ser definidas por altos valores de S. Dessa forma, dos pixels pertencentes à região indicada pela primeira máscara, extrai-se o valor de S e cria-se uma nova máscara contendo todos os pixels com S maior que 150, valor determinado com imagens de referência. Em seguida, essa máscara é somada à segunda máscara, resultando em uma terceira máscara mais completa.

Em seguida, deseja-se recuperar os pixels de cores mais escuras. Cores como preto e cinza não possuem um valor de H bem definido. No caso do preto, qualquer matiz associada a um baixo brilho pode resultar nessa cor; no caso do cinza, a baixa saturação faz com que a matiz tenha pouca relevância. Dessa forma, essas cores podem acabar sendo filtradas quando se utiliza apenas o valor de H como critério de segmentação. Para recuperá-las, os pixels da região indicada pela primeira máscara são convertidos para escala de cinza e cria-se uma nova máscara contendo todos os pixels com valor inferior a 50, valor determinado com imagens de referência. Por fim, essa máscara é somada à terceira máscara, obtendo-se a máscara final, composta pelas faixas com cores distantes da cor do corpo, pelas cores vibrantes e pelas cores escuras. As Figuras 12, 13, 14 e 15 apresentam exemplos da primeira, segunda, terceira e quarta máscara respectivamente obtidas após aplicação de uma imagem no algoritmo.



Figura 12: Primeira máscara.



Figura 13: Segunda máscara.



Figura 14: Terceira máscara.



Figura 15: Máscara final.

No fim dessa etapa, a máscara pode acabar encontrando mais regiões, algumas vezes simplesmente dividindo uma faixa em duas, outras vezes encontrando faixas onde não deveria. Para tratar esse problema, primeiro encontram-se os contornos referentes as máscaras, assim como seus centroides. O algoritmo então verifica, para cada centroide, a distância em relação a outro. Se a distância for muito pequena, o algoritmo entende que os contornos pertencem as mesma faixa e ignora o de menor área. Se ainda tiverem sobrado mais que 3 contornos, o programa seleciona e trabalha somente com os 3 de maior área. A estratégia mostra-se confiável, pois, quando alguma região que não é faixa acaba sendo indicada, ela não é comportada e acaba com uma área pequena. Ao obter os 3 contornos de maior área, maximizam-se as chances de se obter as 3 faixas as quais pretende-se realizar a análise.

Com as máscaras extraídas, segue-se para etapa de orientação do resistor.

3.2.3 Ordenação das faixas

Com a posição das faixas determinadas, o algoritmo precisa encontrar a posição delas em relação ao corpo, para realizar a leitura do valor da resistência corretamente. Para isso, parte-se do pressuposto que a primeira

faixa será mais próxima de alguma das bordas do que a terceira, sendo a do meio a com distância intermediária em relação a alguma das bordas. Portanto, para determinar a posição das faixas, basta calcular a distância do centro de cada faixa (já obtido anteriormente) até as bordas do resistor. Entretanto, o resistor não tem bordas uniformes. Dessa forma, caso o centro de uma faixa esteja acima ou abaixo do centro de outra, a distância de cada uma até a extremidade do resistor será medida em relação a pontos distintos da borda. Consequentemente, uma comparação direta dessas distâncias pode introduzir erros, uma vez que elas não estariam sendo calculadas a partir da mesma referência geométrica.

Para resolver os problemas comentados, utiliza-se da máscara do corpo do resistor para construir uma *Bounding Box*. Esse elemento se trata de um retângulo no qual toda a área do resistor estará contido em seu interior. Assim, calcula-se a distância dos centros até as bordas do retângulo.

Primeiro, determina-se a faixa central. Essa faixa terá a distância intermediária até uma borda, ou seja, não será a maior nem a menor. Portanto, escolhe-se uma das bordas e calcula-se a distância do centro de todas as faixas, adotando a faixa com centro a uma distância intermediária a faixa do meio e as outras como possíveis primeira ou terceira faixa.

Em sequência, para determinar qual faixa é a primeira e qual é a terceira, calcula-se a distância de ambas faixas restantes até as duas extremidades, armazenando a menor distância calculada. Por fim, comparam-se os valores calculados: a faixa cujo valor atribuído for menor será dada como a primeira faixa e a outra, portanto, será dada como terceira faixa. As Figuras 16 e 17 apresentam de forma visual e simplificada o que o algoritmo está fazendo para encontrar a ordem das faixas.

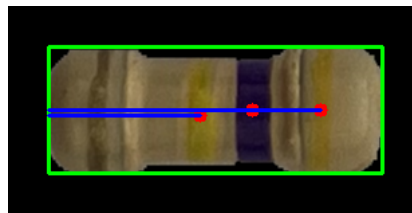


Figura 16: Distâncias até a borda esquerda.

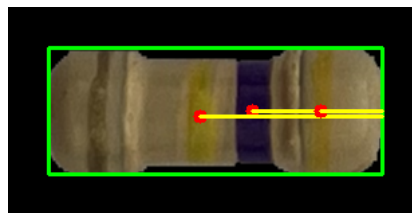


Figura 17: Distâncias até a borda direita.

Analisando as Figuras, percebe-se, de fato, a primeira faixa está mais próxima a borda do que a terceira. Dessa forma, o algoritmo é capaz de decidir corretamente a ordem das faixas e seguir para a decisão das cores.

3.2.4 Decisão das cores e determinação da resistência

Para decidir as cores de cada faixa o algoritmo irá analisar individualmente o contorno de cada faixa obtida. Primeiro, ele pega o centroide de cada contorno, já calculado anteriormente. A partir desse centro, constrói-se um quadrado de 5x5 pixels e toma-se a média do valor HSV desses pixels. Após isso a média é enviada a uma função decisora.

A função construída para determinar a cor da média funciona da seguinte maneira: Primeiro ela separa os valores de H, S e V do pixel para fazer uma análise para cada valor individualmente. A função tem armazenada um valor H de referência para cada cor, obtido a partir da análise do espaço HSV e de teoria de cores. A função então calcula a distância euclidiana entre o H do pixel extraído e o valor H referência de cada cor. Assim, dentre todas as cores avaliadas, considera-se que o pixel pertence àquela cujo valor de H de referência apresenta a menor distância em relação ao valor extraído.

Idealmente, nesse ponto, a função já retornaria a cor do pixel. Entretanto, como comentado, cores como marrom, preto e cinza não dependem só do valor de H. Portanto, é necessário avaliar a possibilidade do pixel ser uma dessas cores, já que também se encontram no código de cores. A cor preta é definida por um V muito baixo, a cor cinza é definida por um V médio e um S muito baixo e a cor marrom é um caso particular da cor vermelha ou laranja: quando o pixel tem o H que tende à uma dessas cores mas tem o brilho muito baixo, na verdade essa cor é marrom. Entretanto, S e V dizem respeito as condições as quais a foto é tirada. Portanto, um V baixo e um S baixo é algo subjetivo e dependerá das condições da foto. Anteriormente, comentou-se sobre as limitações do algoritmo e como ele depende de certas condições da foto para funcionar. Dessa forma, utilizando-se fotos de referência, ajustou-se limiares do decisor que irá verificar os valores S e V do pixel enviado à função; se eles estiverem dentro dos valores ajustados, o pixel perderá a cor anteriormente atribuída e receberá a nova cor dentre as 3 comentadas.

A função então retorna ao algoritmo a cor do pixel enviado como argumento, organizando em uma lista a cor de cada uma das 3 faixas. Com o resultado obtido, uma função que calcula a resistência é chamada. Primeiro ela verifica o valor atribuído a cor com base no próprio código de cores. Em sequência ela aplica a seguinte Equação:

$$R = (C_1 \times 10 + C_2) \times 10^{C_3}, \quad (4)$$

na qual C_1 é o valor atribuído à cor 1, C_2 é o valor atribuído a cor 2 e C_3 é o valor atribuído a cor 3, obtendo o valor numérico da resistência, que é formatado a um formato mais convencional (1 k Ω , 1 M Ω , etc). A função então retorna ao algoritmo o valor formatado e o valor numérico original, sendo o último retornado para ser utilizado na função de validação, que será comentada posteriormente.

3.2.5 Preparação da imagem de saída

No momento atual, o algoritmo já obteve a resistência do componente da imagem. Entretanto, o objetivo do algoritmo é apresentar isso de forma adequada ao usuário e não simplesmente imprimir o valor no terminal. Dessa forma, busca-se uma imagem de saída que será um zoom no resistor, reorientado a posição de leitura, com as faixas do código de cores destacadas e com o valor da resistência escrito acima do resistor.

Para alcançar o resultado desejado, primeiro verifica-se o posicionamento do resistor. Anteriormente, o algoritmo reposicionou o resistor para que ele ficasse na horizontal. Entretanto, essa ação pode colocar o resistor

na posição correta de leitura ou na posição contrária. Para contornar isso o algoritmo usa agora a faixa que sabe ser a primeira. Sabe-se que o código de cores é lido da esquerda para a direita. Portanto, a primeira faixa deve também ser a mais à esquerda das 3. Anteriormente o algoritmo já calculou a distância dessa faixa para as duas extremidades do algoritmo. Dessa forma, basta verificar qual distância é a menor: se for a distância até a borda esquerda, o algoritmo não faz nenhuma operação de reorientação, se for até a borda da direita, o algoritmo espelha a imagem, o contorno do resistor e os contornos das faixas, colocando assim o resistor na posição correta de leitura.

Na sequência, prepara-se o zoom. Se a imagem não for espelhada pelo processo anterior, utiliza-se da *Bounding Box* calculada anteriormente para determinar área do zoom. Caso contrário, outra *Bounding Box* é calculada. O enquadramento não será exatamente na área do elemento, mas com as dimensões ligeiramente ajustadas, utilizando as dimensões da caixa como referência, para que seja possível ver o um pouco o fundo da imagem além do resistor.

Em seguida, para melhor apresentação, obtém-se também as *Bounding Boxes* das faixas coloridas, utilizando os contornos (possivelmente espelhados). Os elementos são diretamente impressos sobre a imagem na cor azul.

Por fim, utilizando-se da biblioteca PIL, escreve-se "Resistor de $x\Omega$ " sobre a imagem. A escolha da biblioteca PIL, nesse caso, se deu pois o opencv não permite a escolha de uma fonte que contenha o caractere " Ω ". Assim, após todo o processamento, uma imagem como a Figura 18 é devolvida ao usuário.

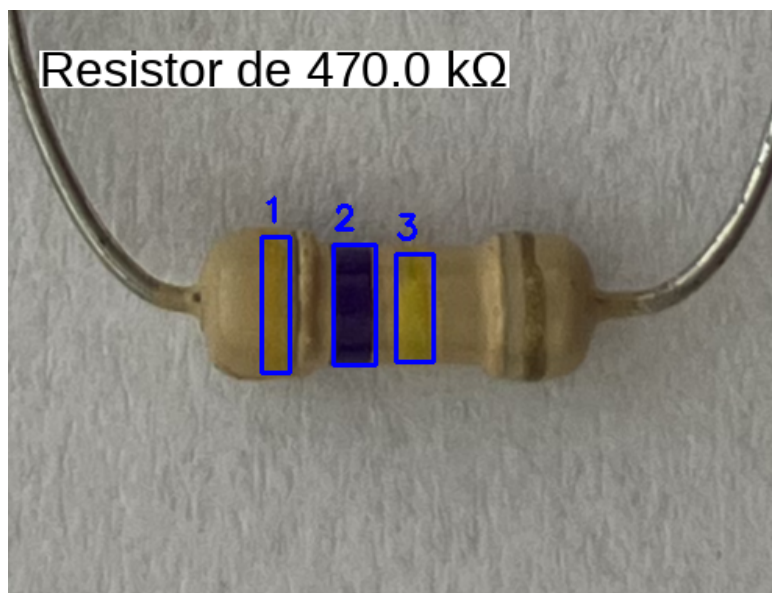


Figura 18: Exemplo de imagem que o algoritmo retorna ao usuário.

O projeto foi carregado no GitHub, podendo ser acessado [aqui](#).

4 Resultados

Ao longo de todo o desenvolvimento do projeto, diversas imagens de diversos resistores foram fotografadas. Buscou-se sempre respeitar as condições colocadas como obrigatórias para funcionamento do algoritmo. Ao fim, eliminou-se as imagens usadas para calibração e criou-se um banco de dados de 58 imagens para validação do algoritmo. Junto às imagens, colocou-se um CSV contendo o nome dos arquivos das fotografias e a resistência do componente naquela fotografia.

Anteriormente, comentou-se que a função que calcula a resistência também retorna o valor puramente numérico. Esse valor puramente numérico é utilizado no algoritmo de validação. Esse programa roda, para todas as imagens dentro do banco de dados, o algoritmo desenvolvido. Ele encerra a execução no momento em que determina a resistência, comparando esse valor com o valor dentro do CSV atribuído a fotografia analisada naquela iteração do algoritmo. Ao fim é retornado o número de falhas, de acertos e a taxa de sucesso do programa.

Aplicando o algoritmo ao banco de 58 fotos comentados, obteve-se uma taxa de sucesso de aproximadamente 86%, acertando 50 das 58 imagens. O resultado obtido mostrou-se satisfatório. Esperava-se que o programa não apresentasse uma taxa acima de 80% devido a complexidade do desafio imposto ao buscar um processamento totalmente livre de técnicas de aprendizado de máquina.

5 Conclusão

Neste trabalho foi desenvolvido um algoritmo capaz de determinar o valor da resistência de resistores a partir de imagens, utilizando exclusivamente técnicas clássicas de visão computacional. Os resultados obtidos demonstraram que a estratégia proposta foi capaz de identificar corretamente a resistência na maioria dos casos avaliados, alcançando uma taxa de acerto de aproximadamente 86%.

Dessa forma, conclui-se que a abordagem adotada é viável para a aplicação proposta, além de reforçar a eficácia de técnicas tradicionais de processamento de imagens na solução de problemas práticos sem a necessidade de métodos de aprendizado de máquina.

Referências

- [1] Muhammed Fatih Demir, Amir Yavariabdi, Aysenur Cankirli, Engin Mendi, Begum Karabatak, and Huseyin Kusetogullari, “Real-time resistor color code recognition using image processing in mobile devices,” in *2018 IEEE International Conference on Intelligent Systems (IS)*. 2018, pp. 26–30, IEEE.
- [2] OpenCV Development Team, “Opencv-python tutorials documentation,” <https://docs.opencv.org>, 2024, Acessado em 2026.
- [3] Tilo Strutz, “The distance transform and its computation: An introduction,” *arXiv preprint arXiv:2106.03503*, 2023.
- [4] Márcio Cherem Schneider and Carlos Galup-Montoro, *CMOS Analog Design Using All-Region MOSFET Modeling*, Cambridge University Press, 1st edition, 2010.